London South Bank University

ENGINEERING PORTFOLIO

/ 2025 – 26

COMPUTER PROGRAMMING

C++ / Numerical Methods / Algorithms



First-Year Mechanical Engineering Laboratory Portfolio

From fundamental programming to engineering-focused computation

$Q = mc\Delta T$ • $\sin\theta$ • $\pi \approx \Sigma$

C++ | Simulation | CSV

BOKHTEAR UDDIN BADHON

BEng (Hons) Mechanical Engineering | LSBU

SEEKING 2027 INDUSTRIAL PLACEMENT

Project overview

A structured progression through programming and computational problem-solving

PROJECT CONTEXT

Computer Programming laboratory logbook

Module: MED_4_SMM — Solid Mechanics and Materials

Submission: 15 December 2025

University: London South Bank University

PORTFOLIO OBJECTIVE

Demonstrate applied C++ fundamentals, numerical calculations, algorithms, data handling and object-oriented programming through documented lab exercises and outputs.

ENGINEERING MINDSET → Define the problem • Build the logic • Run the code • Interpret the result

Programming foundations

Building reliable input, processing and output workflows



Lab Exercise Week 01
Exercise 01

```
/*****************************************************************************
Welcome to GDB Online.
GDB online is an online compiler and debugger tool for C, C++, Python, Java, PHP,
C#, OCaml, VB, Swift, Pascal, Fortran, Haskell, Objective-C, Assembly, HTML, CSS,
Code, Compile, Run and Debug online from anywhere in world.
*****************************************************************************/
#include <iostream>
#include <string>

using namespace std;

int main() {
    string myName = "Bokhtear Uddin Badhon";
    cout << myName << endl;
    cout << "I am learning to code with C++" << endl;

    return 0;
}
```

```
Bokhtear Uddin Badhon
I am learning to code with C++

...Program finished with exit code 0
Press ENTER to exit console.
```

Conclusion

I successfully created and executed a simple C++ program that displays my name and a short message on the screen. The program compiled and ran without any errors, confirming that the basic structure of a C++ program was correctly implemented. This exercise helped me understand how output is displayed in the console using C++.

Logbook Week 01: first C++ console program

IMPLEMENTATION

- main() program structure
- #include and standard libraries
- Variables, strings and cout
- Arithmetic and formatted output
- Compilation and console execution

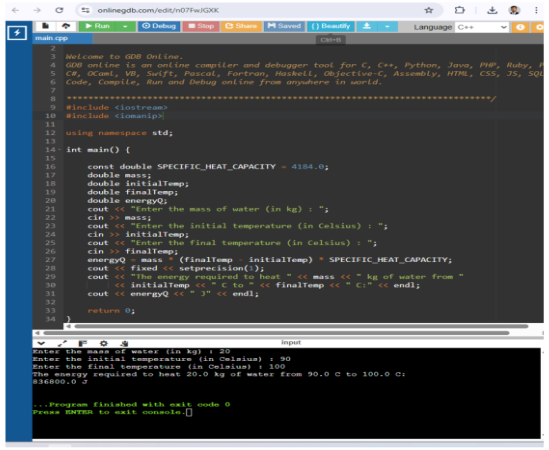
Takeaway: structured, readable code is the foundation of engineering computation.

Engineering calculations in C++

Translating physical equations into executable calculations

- Displaying time in a standard HH:MM:SS format

Exercise 05



Conclusion

This program correctly calculated the amount of energy required to heat a given mass of water between two temperatures. By taking user input and applying a scientific formula, the program produced a realistic and meaningful result. The output was clearly formatted, showing that both the calculation and presentation were handled successfully.

I Learned

Week 02: calculating heat energy from user inputs

PHYSICS → CODE

Energy to heat water: $Q = mc\Delta T$

Inputs: mass and temperature change

Method: multiply by specific heat capacity

Output: energy in joules

Skills: units, constants, numeric types, input/output

Mechanical engineering link: computational evaluation of thermodynamic relationships.

Decision logic and validation

Building programs that respond to valid and invalid inputs

- Formatting numerical output to one decimal place
- Writing descriptive output that explains the calculation

Lab Exercise Week 03

Exercise 01



Conclusion

Week 03: divisibility checks and division-by-zero handling

WHY IT MATTERS

- if / else decisions
- Modulus (%) for divisibility
- Guarding against division by zero
- Communicating clear outcomes
- Defensive programming before computation

Engineering principle: validate inputs before trusting calculated results.

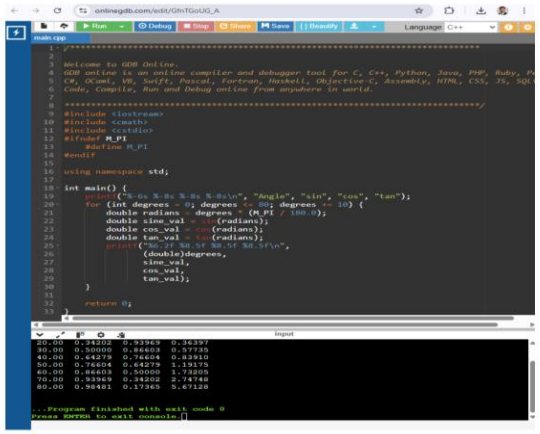
Loops and mathematical functions

Automating repeated calculations and structured numerical output

New Concepts Used

- Logical operators &&, ||, and !=
- Boolean variables to store condition results
- Custom function to convert boolean values to strings
- Output alignment using setw()
- Structured evaluation of multiple related conditions

Lab Exercise Week 04
Exercise 01



Conclusion

Week 04: trigonometric table, 20° to 80°

TECHNICAL APPROACH

- Convert degrees to radians
- Evaluate sin(), cos() and tan()
- Iterate through angles using a loop
- Display aligned tabular results
- Reuse mathematical library functions

Application: angles and trigonometric relationships underpin forces, motion and geometry.

Monte Carlo simulation

Using random sampling to investigate probability

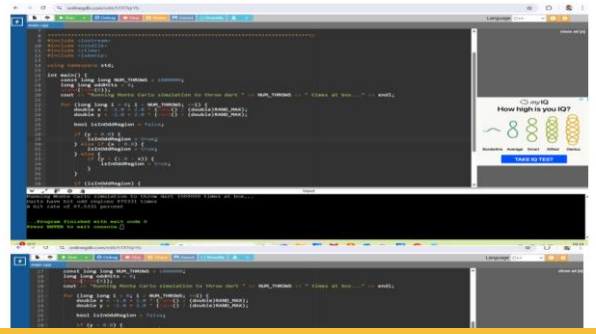
From completing this exercise, I learned how to:

- Retrieve system time using built-in C++ functionality
- Perform multiple step-by-step calculations on large values
- Convert raw numerical data into a meaningful real-world format
- Use formatting tools to display time clearly and consistently

New Concepts Used

- time(0) to access the current system time
- Constants to represent fixed time values (seconds per minute/hour/day)
- Modulus (%) and division to extract time components
- Output formatting using setw() and setfill()
- Displaying time in standard HH:MM:SS notation

Exercise 05



Week 04: virtual dart experiment and sampling output

SIMULATION WORKFLOW

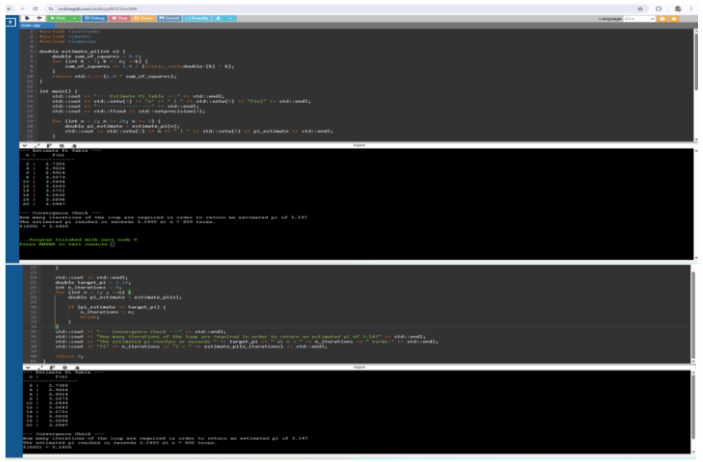
1. Generate random sample points
2. Test whether each point meets a condition
3. Count successful events
4. Calculate the percentage
5. Interpret the approximate probability

Engineering relevance: simulation-based estimation and computational experimentation.

Numerical approximation of π

Exploring how additional iterations affect convergence

Exercise 02



Conclusion

This program successfully estimated the value of π using a mathematical series and displayed the results in a formatted table. It also analysed how the estimate improves as more terms are included and determined when the approximation reached a chosen accuracy level. The output demonstrated that the algorithm converges towards the true value of π as expected.

I Learned

- Implement a mathematical series in code
- Use loops to gradually improve an approximation

Week 06: series approximation and convergence checks

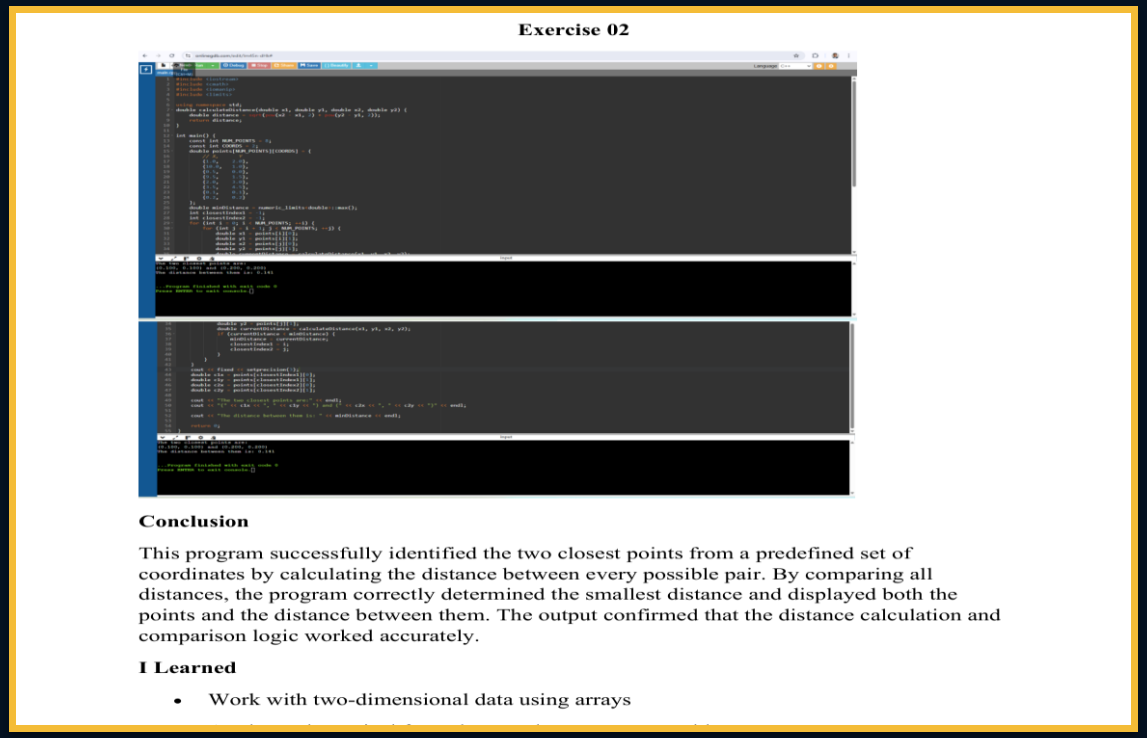
NUMERICAL THINKING

- Implement a mathematical series
- Accumulate terms using loops
- Compare estimates as iterations increase
- Check a selected accuracy threshold
- Format numerical results for comparison

Engineering relevance: accuracy, convergence and computational approximation.

Arrays and coordinate geometry

Using data structures to solve a geometric comparison problem



Exercise 02

Conclusion

This program successfully identified the two closest points from a predefined set of coordinates by calculating the distance between every possible pair. By comparing all distances, the program correctly determined the smallest distance and displayed both the points and the distance between them. The output confirmed that the distance calculation and comparison logic worked accurately.

I Learned

- Work with two-dimensional data using arrays

Week 08: closest pair of points from coordinate data

ALGORITHM

- Store coordinate pairs in a 2D array
- Calculate Euclidean distances
- Compare all point combinations
- Track the minimum distance
- Return the closest point pair

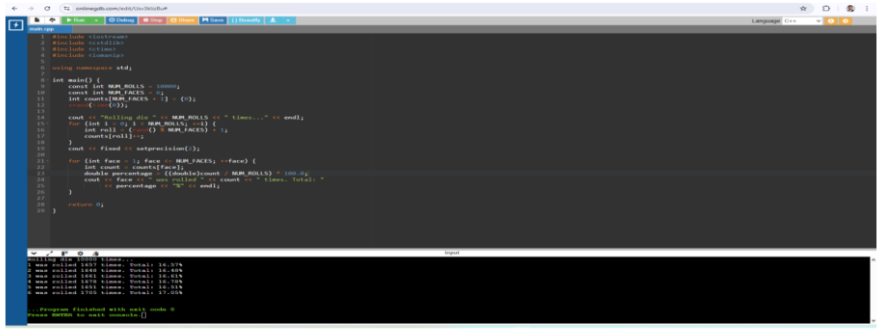
Engineering relevance: geometric reasoning, spatial data and distance calculations.

Statistical simulation and results

Turning repeated trials into interpretable frequency data

- Returning and using calculated values from functions

Exercise 03



Conclusion

This program successfully simulated rolling a six-sided die a large number of times and recorded how often each face appeared. The results were displayed both as counts and percentages, showing that the outcomes were distributed fairly evenly. This confirmed that the random number generation and counting logic worked correctly.

I Learned

- Simulate repeated random events using loops
- Store frequency data efficiently using arrays
- Calculate percentages from collected data
- Interpret simulation results to check randomness

Week 08: simulated die rolls, counts and percentages

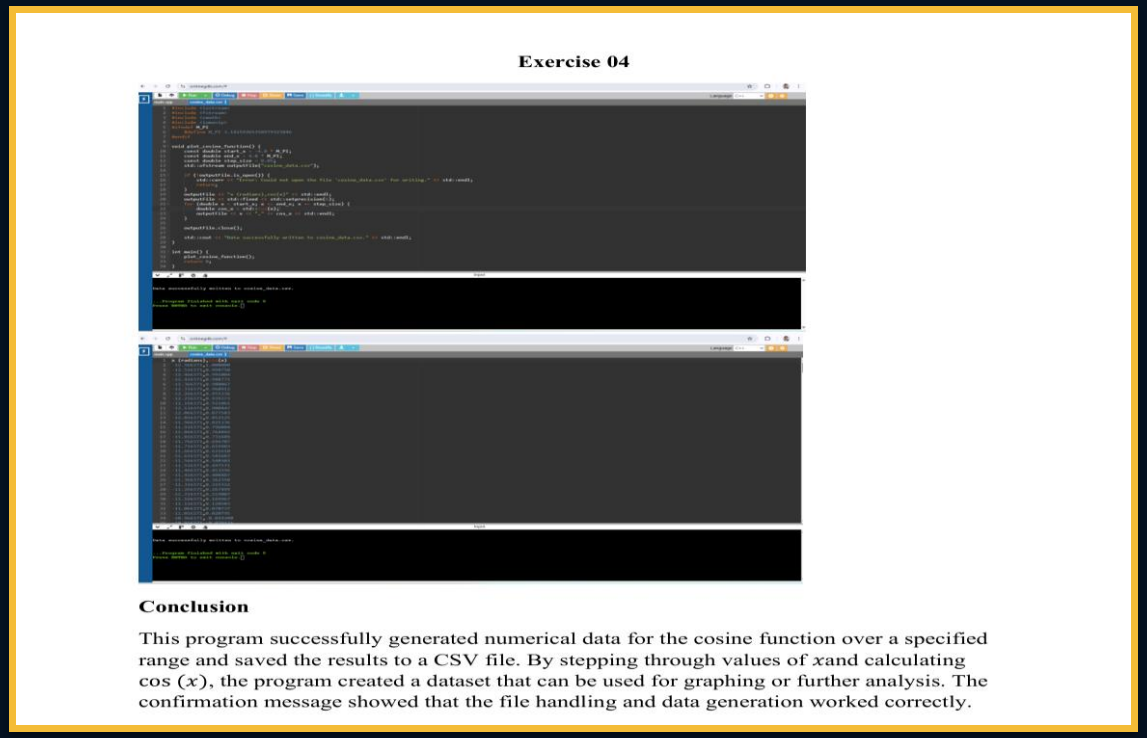
DATA PROCESSING

- Generate repeated random events
- Store event counts in arrays
- Compute frequency percentages
- Compare outcome distributions
- Present structured numerical output

Transferable skill: summarising simulation results rather than only producing code.

CSV generation and data export

Creating reusable numerical datasets beyond the console



Exercise 04

Conclusion

This program successfully generated numerical data for the cosine function over a specified range and saved the results to a CSV file. By stepping through values of $x$and calculating cos (x), the program created a dataset that can be used for graphing or further analysis. The confirmation message showed that the file handling and data generation worked correctly.

Week 08: generating cosine values and saving CSV output

DATA PIPELINE

INPUT: numerical x-range and step size

PROCESS: calculate cos(x) repeatedly

EXPORT: write structured rows using ofstream

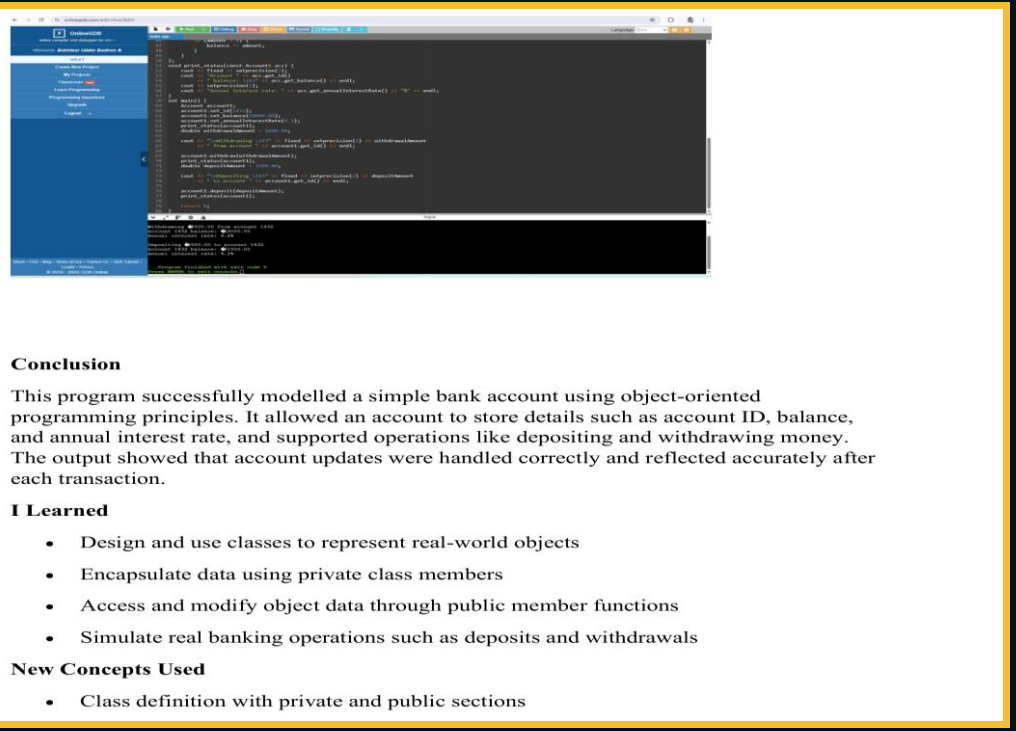
USE: inspect, graph or analyse in a spreadsheet

Skills: file handling, precision and reproducibility

Engineering relevance: a foundation for simulation data export and post-processing.

Object-oriented programming

Organising code around objects, attributes and operations



Conclusion

This program successfully modelled a simple bank account using object-oriented programming principles. It allowed an account to store details such as account ID, balance, and annual interest rate, and supported operations like depositing and withdrawing money. The output showed that account updates were handled correctly and reflected accurately after each transaction.

I Learned

- Design and use classes to represent real-world objects
- Encapsulate data using private class members
- Access and modify object data through public member functions
- Simulate real banking operations such as deposits and withdrawals

New Concepts Used

- Class definition with private and public sections

Week 09: bank-account class and member functions

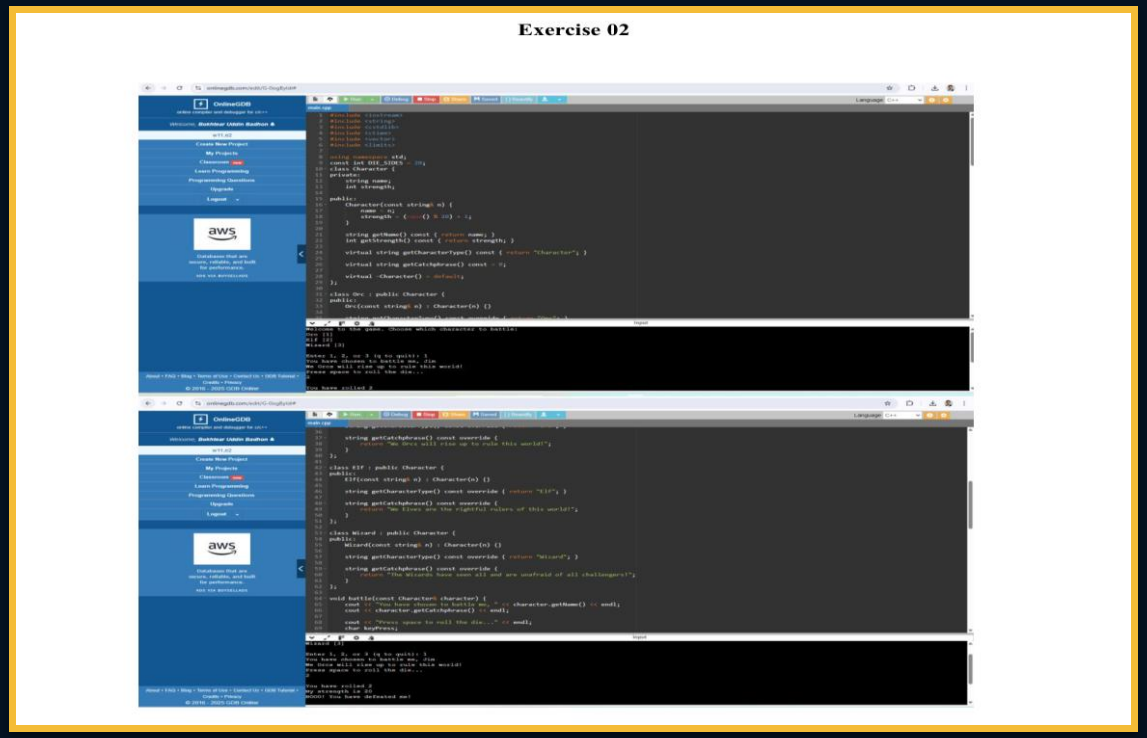
SOFTWARE DESIGN

- Classes and objects
- Private data and encapsulation
- Getters and setters
- Constructors and member functions
- Separating an object’s state from its operations

Transferable skill: structuring more complex computational models into reusable components.

Algorithms and performance

Learning to select, implement and compare computational methods



Later logbook: programming evidence and algorithm work

SKILLS DOCUMENTED

- Selection, insertion and bubble sorting
- Search techniques
- Arrays and dynamic memory
- Functions and program decomposition
- Timing-based performance comparison

These are learning exercises, not industrial benchmarks.

Engineering relevance: algorithm choice affects computational cost and maintainability.

Development approach

How the lab exercises built a repeatable problem-solving process

01

UNDERSTAND

Identify inputs, outputs and constraints

02

IMPLEMENT

Translate mathematics or logic into C++

03

TEST

Compile, execute and inspect console output

04

REFLECT

Document conclusions and new concepts

REFLECTION

The logbook records code evidence, execution results, conclusions and learning points across successive lab weeks.

Technical skills and placement relevance

What these first-year exercises demonstrate to a prospective employer

TECHNICAL FOUNDATION

- C++ programming and debugging
- Mathematical and scientific calculations
- Loops, conditions, functions and arrays
- Numerical approximation and simulation
- CSV output and data processing
- Object-oriented programming

ENGINEERING BEHAVIOURS

- Analytical problem-solving
- Step-by-step implementation
- Input validation and checking
- Clear technical documentation
- Interpretation of numerical results
- Willingness to learn and improve

Seeking a 2027 industrial placement to develop these foundations in practical engineering work.

Thank you / Let’s connect

Computer Programming • Mechanical Engineering • Continuous Learning



BOKHTEAR UDDIN BADHON

BEng (Hons) Mechanical Engineering

London South Bank University

INTERESTED IN 2027 INDUSTRIAL PLACEMENTS

Engineering computation • Problem-solving • Learning by doing